

FoldQuantVLA: Native Low-Bit Quantization of Vision-Language-Action Models via Consistent Folding

Hung T. Ho¹, Khanh D. Nguyen¹, Quang D. Nguyen¹, Thanh Q. Duong¹,
Ngan Le², Meng Guo³, Vien A. Ngo^{1,4}, An T. Le^{1,4,5,*}

Abstract—Low-bit vision-language-action inference must reduce observation-to-action latency while preserving robot behavior. We present FoldQuantVLA, a post-training quantization framework that carries a consistent activation representation through calibration, weight rounding, and native integer execution. It combines channel scaling and block Hadamard transforms with dynamic per-token quantization, without policy retraining. Custom TensorRT plugins execute projections in both the language backbone and iterative action expert with four-bit weights and activations (W4A4) on Ada GPUs and Jetson AGX Orin. Evaluation spans LIBERO, SimplerEnv, and two robot platforms. Across three GR00T checkpoints and $\pi_{0.5}$, W4A4 achieves 1.20–1.33 $\times$ speedups over floating-point TensorRT on Orin and 1.25–1.52 $\times$ on desktop. Retaining language attention-output and feed-forward down projections at eight bits (W8A8) improves held-out action fidelity on all four checkpoints. Across four real-robot tasks, this configuration raises observed GR00T N1.7 success from 80.0% with uniform W4A4 to 92.5% over 80 trials per configuration, with a measured additional Orin latency of 1 ms.

Index Terms—Vision-language-action models, post-training quantization, robot deployment.

I. INTRODUCTION

The vision-language-action (VLA) policies evaluated here combine visual processing, a language backbone, and an iterative denoising action expert [1], [2]. Both backbone and expert lie on the latency-critical path. Quantization must also preserve closed-loop behavior: action errors change subsequent observations.

Post-training quantization reduces precision without retraining. Channel scaling, rotations, and activation-aware rounding address large activations [3]–[8], but their integration requires consistent calibration and execution coordinates. We assign one change of basis to each shared activation site, fold its inverse into every consuming weight, and round after fixing that basis.

We study uniform W4A4 projections and a selective-INT8 configuration for sensitive language output projections (Fig. 1). Floating-point TensorRT controls measure gains beyond compilation. Held-out action fidelity screens numerical degradation, while simulator and physical-robot trials assess behavior. Project and code links accompany the paper.¹

¹VinRobotics, Vietnam. ²AICV Lab, EECS Department, University of Arkansas. ³School of Advanced Manufacturing and Robotics, Peking University, China. ⁴Center for AI Research, VinUniversity, Vietnam. ⁵Intelligent Autonomous Systems, TU Darmstadt, Germany. *Corresponding author: An T. Le (an@robot-learning.de).

¹Project: <https://review-artifact-27f4.github.io/foldquantvla/>. Code repository: <https://github.com/cair-vinuni/FoldQuantVLA>.

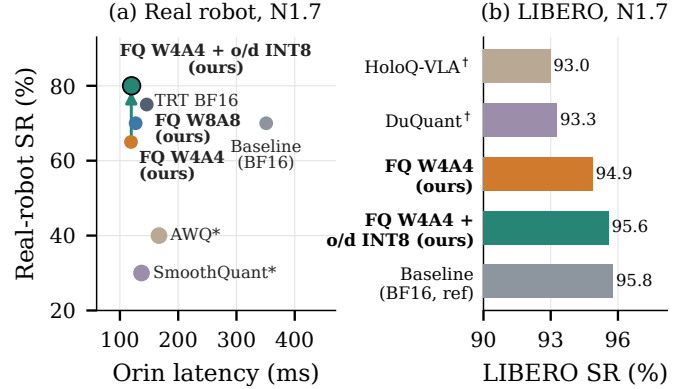


Fig. 1. FoldQuantVLA at a glance. (a) N1.7 on Orin: observation-to-action latency versus real-robot success rate (SR) on block-stacking task T4, with 20 trials per configuration. Upper left is better: selective INT8 raises observed T4 success from 65% to 80% for 1 ms. *ModelOpt SmoothQuant (W8A8) and AWQ (W4A16): T4 only (Table V). (b) LIBERO SR on N1.7; †: our emulated ports (Table I). Baseline (ref): BF16 PyTorch; FQ: FoldQuantVLA; TRT BF16: floating-point TensorRT; o/d INT8: o_proj/down_proj at W8A8.

Our contributions are:

- **Consistent folding.** A folding contract that assigns one transformed coordinate system to each shared activation site and carries it through calibration, rounding, and native low-bit execution.
- **A W4A4 recipe for VLA policies.** Channel scaling, block Hadamard rotations via the fast Walsh–Hadamard transform (FWHT), and per-token quantization avoid timestep-indexed scale tables. Proposition 2 gives conditions for step-invariant normalized projection error.
- **Selective INT8 at o_proj and down_proj.** W8A8 at these sites improves held-out fidelity on all four checkpoints and N1.7 robot success from 80.0% to 92.5% for 1 ms on Orin.
- **Deployable native low-bit engines.** TensorRT plugins execute W4A4 projections in both backbone and expert, achieving 1.20–1.33 $\times$ speedups over floating-point TensorRT on Orin, with evaluation on LIBERO, SimplerEnv, ALOHA, and SO-101.

II. RELATED WORK

Transforms and rounding. SmoothQuant scales channels [3]; SliceGPT formalizes computational invariance, QuaRot exploits it for quantization, and SpinQuant learns

rotations [4], [5], [9]. DuQuant, SmoothRot, FlatQuant, and OSTQuant combine or optimize equivalent transforms [6], [7], [10], [11]. GPTQ approximately optimizes weight rounding [8], while AWQ protects salient channels [12]. Our focus is coordinate consistency across shared consumers and the executable VLA graph.

Low-bit execution. Atom and QServe couple quantization with serving kernels [13], [14]; SVDQuant targets four-bit diffusion [15]. MixLLM and HAWQ-V3 study mixed precision [16], [17]. We retain two language projection types at INT8, with integer GEMMs in both configurations at batch 1.

VLA quantization. QuantVLA, QVLA, ActQuant, and DyQ-VLA study calibration, precision allocation, and temporal structure [18]–[21]. HoloQ-VLA combines SVD–Hadamard rotations and per-step action-head scales, with native INT4 in both $\pi_{0.5}$ backbone and expert [22]. Section V separates our emulated N1.7 recipe comparisons from reported $\pi_{0.5}$ results and hardware-specific engine measurements.

III. METHOD

FoldQuantVLA first fixes a shared activation transform and folds its inverse into each consuming weight. It then rounds the folded weights and executes the matching activation transform before integer multiplication. Algorithm 1 summarizes these offline and runtime stages.

A. Scope and execution model

A policy maps an observation o , instruction ℓ , and initial denoising noise ϵ to an action chunk,

$$a = \pi(o, \ell, \epsilon) = \mathcal{D}(\mathcal{E}(\mathcal{L}(\mathcal{V}(o), \ell), \epsilon)) \in \mathbb{R}^{H \times d_a}, \quad (1)$$

where $\mathcal{V}$ is visual processing, $\mathcal{L}$ the language backbone, $\mathcal{E}$ the iterative action expert, and $\mathcal{D}$ the action decoder. The chunk contains H actions of dimension d_a , and proprioception enters $\mathcal{E}$ implicitly. The expert runs n_τ denoising iterations: four for GR00T, and ten for $\pi_{0.5}$. The controller executes the first $K \leq H$ actions before replanning.

We quantize attention and feed-forward projection matrix multiplications (GEMMs) in $\mathcal{L}$ and $\mathcal{E}$. A token is a feature vector; its coordinates are channels. $W_b A_b$ denotes weight/activation bit widths. Vision, non-projection attention operations, normalization, residual arithmetic, and the action decoder remain floating point. In W4A4 engines, GR00T’s adaptive layer-normalization modulation uses weight-only INT4 with floating-point activations. W4A4 and W8A8 projections use per-token activation scales and per-output-channel weight scales. An N1.6 ALOHA audit over 30 observations motivates per-token scaling: at the worst language layer (`down_proj`), activation spread $\max|x|/\text{median}|x|$ is $557,518\times$ per tensor versus $8,650\times$ per token. The external per-tensor control also differs in calibration and execution (Table IV). Figure 2 illustrates one projection.

B. A consistent fold

For one projection $y = Wx + b$, an invertible transform T gives $z = Tx$ and $\tilde{W} = WT^{-1}$, so $\tilde{W}z + b = Wx + b$ before

quantization. The same identity must hold for every projection that reuses z .

Unroll one policy call into a finite directed acyclic graph with operators $x_v = f_v((x_u)_{u \in \text{pa}(v)})$, where $\text{pa}(v)$ indexes the inputs. Site v holds one activation tensor x_v with d_v channels, shared by its consumers. A fixed invertible $T_v \in \mathbb{R}^{d_v \times d_v}$ acts tokenwise, giving $z_v = T_v x_v$; graph and engine boundaries use identity transforms.

Proposition 1 (Consistent folding). *Define $\tilde{f}_v((z_u)_{u \in \text{pa}(v)}) = T_v f_v((T_u^{-1} z_u)_{u \in \text{pa}(v)})$. The transformed graph computes the same external function in exact arithmetic, as in equivalent-transform methods [4], [9]. An affine operator $x_v = \sum_{u \in \text{pa}(v)} W_{vu} x_u + b_v$ becomes*

$$\tilde{W}_{vu} = T_v W_{vu} T_u^{-1}, \quad \tilde{b}_v = T_v b_v. \quad (2)$$

Proof. In topological order, substituting $z_u = T_u x_u$ gives $z_v = T_v x_v$. Identity output transforms preserve the external function. Substitution into the affine operator gives (2). $\square$

Every projection consuming site v must fold T_v^{-1} into its weights. Query/key/value projections can share one transformed input, as can feed-forward up/gate branches, provided activation bit widths and clipping ratios also match. A consumer using $T_i \neq T_v$ incurs pre-rounding error $W_i(T_i^{-1} T_v - I)x_v$. Such a mismatch lowers N1.6 action-module cosine to 0.9923 (Table IV).

C. Transforms supported by the runtime

A transform must fit the runtime graph. Every transform acts on a projection input and projection outputs keep $T = I$, so residual additions, RoPE, and attention dot products are unchanged; general rotations cannot pass through nonlinearities. We restrict projection-input transforms to

$$T_v = D_v^\circ R_v D_v^i, \quad R_v^\top R_v = I, \quad (3)$$

where D_v^i and D_v° are positive diagonal matrices. With unchanged output coordinates, a consumer stores $\tilde{W}_{iv} = W_{iv} (D_v^i)^{-1} R_v^\top (D_v^\circ)^{-1}$.

Fold-before uses $(D_v^i, D_v^\circ) = (S_v^{-1}, I)$, where $S_v = \text{diag}(s_{v,c})$ is calibrated in the original coordinates, giving $\tilde{W} = W S_v R_v^\top$. *Fold-after* uses (I, P_v^{-1}) , where P_v is positive diagonal and calibrated after rotation, giving $\tilde{W} = W R_v^\top P_v$. Both preserve the unquantized projection but expose different coordinates to rounding. In general, $R_v S_v R_v^\top$ is dense and cannot be represented by a diagonal P_v .

Fold-before is the FWHT default because division fuses into the load loop; fold-after needs an extra pass. At language SmoothQuant sites, let $a_{v,c}$ be the maximum absolute calibration activation in channel c , and $w_{v,c}$ the largest input-channel weight magnitude across shared consumers. For $\alpha \in [0, 1]$, set $s_{v,c} = a_{v,c}^\alpha / w_{v,c}^{1-\alpha}$, with numerical floors and bounded positive scales [3]. Statistics use the coordinates where scaling is applied; at $\alpha = 1$, unclamped scales equalize calibrated ranges before rotation.

The language path uses the normalized block Hadamard transform $R_v = I_{d_v/\beta} \otimes \beta^{-1/2} \mathcal{H}_\beta$, with $\beta = 64$ and $\beta \mid d_v$.

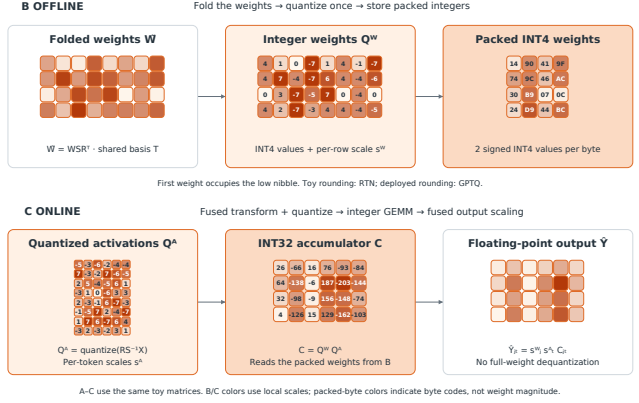
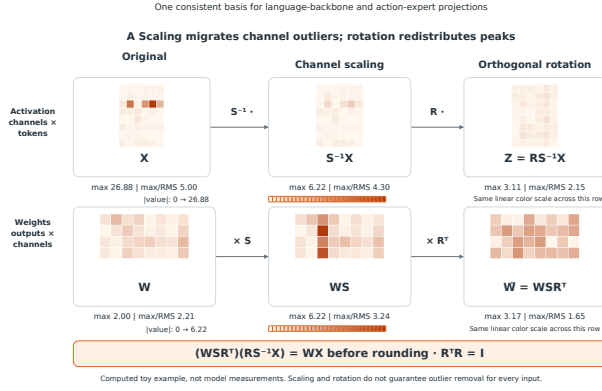


Fig. 2. Fold offline, execute in low bits (toy matrices, not model measurements). (A) Channel scaling S^{-1} migrates activation outliers into the weights and the orthogonal rotation R mixes the scaled channels; $(WSR^T)(RS^{-1}X) = WX$ before rounding, with activation tokens in the columns of X . (B) Offline, the folded weight $\hat{W} = WSR^T$ is rounded once to INT4 with per-output-channel scales and nibble-packed. (C) Online, the fused prologue quantizes $RS^{-1}X$ per token, the integer GEMM accumulates in INT32, and the output is rescaled by $s_j^W \delta_t$ (with $s_t^A = \delta_t$ in the figure) without dequantizing the weights.

Since $\mathcal{H}_\beta \mathcal{H}_\beta^T = \beta I$, normalization ensures $R_v^{-1} = R_v^T$. Here $\mathcal{H}_\beta$ is the Sylvester Hadamard matrix with entries ± 1 , and $\otimes$ repeats it over channel blocks. The FWHT costs $O(d_v \log \beta)$ and preserves the squared norm; it need not reduce the peak of every input. The runtime prologue applies this transform. The action expert uses the same block Hadamard by default, applied as an FWHT, or, optionally, a stored composite rotation (zigzag channel permutation and per-block SVD-Hadamard matrices).

D. Normalization gains

For a projection following root-mean-square (RMS) normalization, write $x_v = \Gamma_v \bar{h}_v$, where $\Gamma_v = \text{diag}(\gamma_v)$ contains the learned gain and $\bar{h}_v$ is normalized without it. Then

$$z_v = D_v^o R_v (D_v^i \Gamma_v) \bar{h}_v. \quad (4)$$

The product $D_v^i \Gamma_v$ replaces the learned gain without moving a scale through the RMS denominator. This requires a preceding gain, absent at the evaluated `o_proj` and `down_proj` inputs. Other exact folds remain possible: a diagonal scale before `down_proj` can be absorbed into the linear up branch of a gated MLP without crossing the gate nonlinearity.

E. Offline parameters and runtime computation

Scales, rotations, folded weights, and clipping ratios are fixed at build time. The activation transform remains runtime work. Folding absorbs fixed parameters into neighboring operators; fusion reduces launches and intermediate tensors (Fig. 3).

At a fixed site (index v suppressed), token $z_t = Tx_t$ is quantized to the floating-point reconstruction

$$\hat{z}_t = \delta_t \text{clip}(\text{round}(z_t/\delta_t), -q, q), \quad \delta_t = \frac{r \|z_t\|_\infty}{q}, \quad (5)$$

where $b_a \in \{4, 8\}$, $q = 2^{b_a-1} - 1$, $r \in (0, 1]$ is a clipping ratio calibrated only at language INT4 sites ($r = 1$ elsewhere), and rounding and clipping are elementwise. We define $\hat{z}_t = 0$

when $z_t = 0$. Each token supplies its own scale, without a timestep-indexed scale table.

Proposition 2 (Amplitude invariance). *In exact arithmetic, (5) is positively homogeneous: $(\lambda \hat{z}_t) = \lambda \hat{z}_t$ for $\lambda > 0$. For a fixed transform T and fixed quantized folded weights $\hat{W}_T$, with the same bias in both projections, the residual $e_T(u) = \hat{W}_T \hat{T}u - Wu$ therefore satisfies $e_T(\lambda u) = \lambda e_T(u)$. If inputs are almost surely nonzero and $u_\tau / \|u_\tau\|_2$ follows the same distribution ν at every denoising step, then*

$$\mathcal{L}_\tau(T) = \mathbb{E} \left[\frac{\|e_T(u_\tau)\|_2^2}{\|u_\tau\|_2^2} \right] \quad (6)$$

is independent of τ . If the directional-distribution assumption holds for every candidate in a common feasible set, any minimizer is shared across steps.

Proof. Scaling z_t by $\lambda > 0$ scales δ_t by λ and leaves the rounding argument unchanged. This gives both homogeneity identities. Writing $u_\tau = \|u_\tau\|_2 v_\tau$ yields $\mathcal{L}_\tau(T) = \mathbb{E}_{v \sim \nu} \|e_T(v)\|_2^2$, independent of τ . $\square$

One engine accepts the model’s timestep conditioning at every iteration without a distributional assumption. Proposition 2 additionally requires stable input directions for step-invariant normalized risk. Dynamic scaling removes amplitude variation; it does not address changing channel distributions [22] or accumulated error [21], [23], [24], nor establish optimality for decoded actions or closed-loop behavior.

F. Rounding and native execution

Collect N_v calibration tokens as columns of $U_v \in \mathbb{R}^{d_v \times N_v}$ and set $Z_v = T_v U_v$; one observation may supply many tokens.

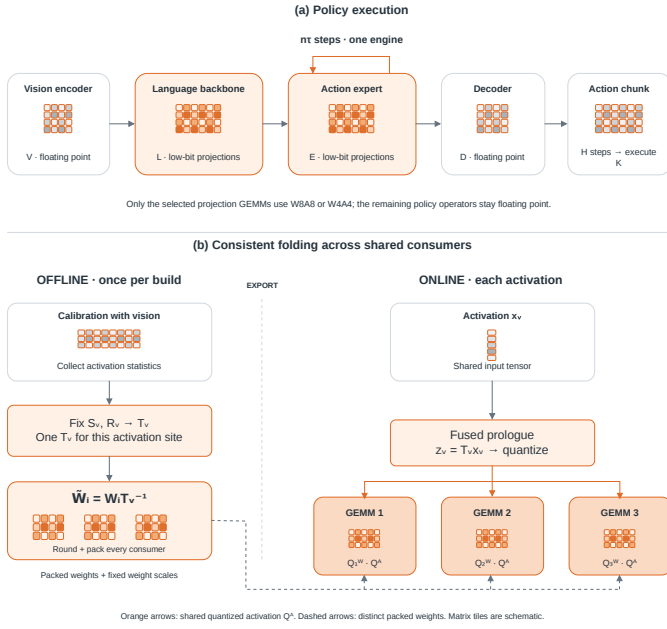


Fig. 3. (a) Policy execution: W8A8/W4A4 projections in $\mathcal{L}$ and $\mathcal{E}$; one expert engine serves n_T steps. The controller executes K of H actions. GR00T’s weight-only modulation is omitted. (b) Shared-input execution: offline, fix T_v , fold $\tilde{W}_i = W_i T_v^{-1}$, round, and pack. Online, transform and quantize once per site; consumers use their own weights and output scales (Algorithm 1).

For four-bit projection weights, GPTQ approximately minimizes reconstruction error in these transformed coordinates:

$$\min_{Q \in \mathcal{Q}} \|(\tilde{W} - Q)Z_v\|_F^2, \quad G_v = Z_v Z_v^T = T_v U_v U_v^T T_v^T, \quad (7)$$

where Q contains representable dequantized weight matrices under the chosen scale layout, and $G_v \in \mathbb{R}^{d_v \times d_v}$ is the unnormalized second moment. GPTQ uses G_v with diagonal damping for numerical stability [8]; eight-bit weights use round-to-nearest (RTN). Calibration through the policy forward pass includes visual conditioning. The objective scores weight rounding alone, leaving activation quantization error outside it [25], [26]. GPTQ improves fidelity over RTN (Table IV).

The W4A4 backend uses packed signed-INT4 weights, dynamically quantized signed-INT4 activations, and INT32 accumulation. Write $\hat{W}_{jc} = s_j^W q_{jc}^W$ and $\hat{z}_{ct} = \delta_t q_{ct}^A$, where q^W, q^A are integer codes, j indexes output channels, and c input channels. The output is

$$\hat{y}_{jt} = s_j^W \delta_t \sum_c q_{jc}^W q_{ct}^A + b_j. \quad (8)$$

Thus rescaling and bias addition are floating point; a zero token gives $\hat{y}_t = b$. W8A8 uses the corresponding integer path and folded parameters. TensorRT 10.3’s built-in INT4 quantization is weight-only [27]; our plugins provide the INT4 activation path. The transform and per-token range reduction execute within our plugin, although plugin boundaries can prevent some TensorRT fusions. HadaCore studies related Hadamard kernels [28].

Algorithm 1 FoldQuantVLA at targeted projection sites

Require: Policy, calibration observations, fold preset, site precisions $p_v \in \{4, 8\}$; $\mathcal{C}(v)$ denotes consumers of site v .

Ensure: Packed weights, fixed scales/transforms, and matching runtime operators.

Offline: once per build

```

1: for each targeted activation site  $v$  do
2:   Collect calibration tokens  $U_v$  through the policy.
3:   Fix shared  $T_v$  from (3) and clipping ratio  $r_v$  in (5).
4:    $Z_v \leftarrow T_v U_v$ ; if  $p_v = 4$ , form  $G_v \leftarrow Z_v Z_v^T$ .
5:   for each consumer  $i \in \mathcal{C}(v)$  do
6:      $\tilde{W}_{iv} \leftarrow W_{iv} T_v^{-1}$ .
7:   Round  $\tilde{W}_{iv}$ : GPTQ with  $G_v$  at four bits, RTN at eight bits.
8:   Store packed codes  $q_{iv}^W$ , output-channel scales  $s_{iv}^W$ , and bias  $b_i$ .
9:   end for
10: end for

Runtime: for each token  $x_t$  at site  $v$ 
11:  $z_t \leftarrow T_v x_t$ .
12: If  $z_t = 0$ , set  $(q_t^A, \delta_t) = (0, 0)$ ; otherwise quantize via (5).
13: for each consumer  $i \in \mathcal{C}(v)$  do
14:    $c_{it} \leftarrow q_{iv}^W q_t^A$  ▷ INT32 accumulation
15:    $\hat{y}_{it} \leftarrow \delta_t (s_{iv}^W \odot c_{it}) + b_i$ .
16: end for

```

Both plugins use CUTLASS v3.6.0. The four-bit GEMM uses an m16n8k64 MMA shape and the eight-bit path uses m16n8k32. Native INT4 runs on Ada and Orin; Section IX documents H100 lowering.

Algorithm 1 uses $p_v = b_w = b_a$ for each site and $\odot$ for elementwise multiplication. It shows the logical transform $T_v x_t$: when a scale is absorbed into normalization via (4), the prologue applies only the remaining factors.

G. Selective INT8 at output projections

FQ W4A4 uses uniform low-bit projections. In **FQ W4A4** + o/d INT8, language attention output (o_proj) and feed-forward down (down_proj) projections use W8A8; the remaining quantized projections use W4A4. Both configurations retain integer GEMMs at every targeted projection.

These two projection types return features to the residual stream. Their inputs lack the normalization gain used in (4), and down_proj has the largest language activation spread in our audit. These observations motivate the precision choice, but do not prove the cause of sensitivity. The same site choice applies to all four checkpoints, without retraining.

QuantVLA keeps action-expert attention projections in floating point [18]. BitNet a4.8 uses eight-bit inputs at attention output and feed-forward down projections, with sparsification, in a 1.58-bit model trained for that format [29]. Our post-training choice uses W8A8 without sparsification and preserves the folding contract. On N1.6/N1.7, o/d INT8 also uses the dense-rotation calibration preset, so comparisons with the main W4A4 rows include that change; N1.5 and $\pi_{0.5}$ change site precision only.

We screen decoded actions against BF16 under shared initial noise, checking coordinate errors and held-out minima because cosine ignores magnitude. Each selected configuration is rebuilt and checked on the assembled engine before closed-loop evaluation.

IV. EXPERIMENTAL PROTOCOL

We evaluate GR00T N1.7, N1.6, N1.5 [30]–[32], and $\pi_{0.5}$ [2]. N1.7 and N1.5 use our four-suite LIBERO fine-tunes, N1.6 a public four-suite fine-tune, and $\pi_{0.5}$ an author-released checkpoint. GR00T uses four denoising steps and action chunks of length 16; the executed prefix is eight actions for N1.7/N1.6 and one for N1.5. For $\pi_{0.5}$, these quantities are ten, ten, and five.

Success tables report percentages (higher is better); bold values mark the best result for each task or checkpoint, including ties. FQ denotes FoldQuantVLA.

Fidelity. P1 checks 32 seeded observations on H100 MIG 3g.40gb against BF16 PyTorch. P2 uses 32 held-out mid-trajectory observations, disjoint from the 128 calibration observations, on RTX 4070 Ti SUPER. Both compare flattened decoded action chunks under shared initial noise. P2 reports minimum cosine similarity (directional agreement) and median maximum absolute coordinate error (in policy action units) over observations.

LIBERO. P3 uses four suites [33], ten tasks per suite, and 20 initial states per task: 800 episodes per arm, seed 7, 520-step cap, on H100 MIG. Table II compares five deployment arms. Tests use paired per-task success differences with a t_{39} reference; the four preset-matched p -values are unadjusted and exceed 0.05.

Latency. Desktop timing uses one run per checkpoint, 60 observation-to-action iterations after ten warmups, on a dedicated RTX 4070 Ti SUPER with TensorRT 10.15. Orin uses Jetson AGX Orin with TensorRT 10.3; both use batch 1. Controls include eager PyTorch and floating-point TensorRT. ModelOpt baselines [34] are included on Orin; desktop ModelOpt/compile values use different timing paths and are excluded.

V. COMPARISON WITH PRIOR W4A4 RECIPES

Table I compares prior W4A4 recipes on LIBERO. Both FoldQuantVLA configurations score above our N1.7 HoloQ-VLA and DuQuant ports. The emulated ports retain their own projection coverage, so these compare complete recipes; the paired differences are not significant.

For $\pi_{0.5}$, o/d INT8 reaches 97.8%, versus HoloQ-VLA’s reported 98.0%. Our long-suite re-run of its released pack gives 94.0% versus the reported 96.0% ($p = 0.44$). HoloQ-VLA also reports native INT4×INT4 execution at $1.03\times$ ($77.08 \rightarrow 75.12$ ms, RTX 4080 SUPER) [22]; FQ W4A4 reaches $2.22\times$ on RTX 4070 Ti SUPER (Fig. 4). Different hardware and baselines preclude a direct latency comparison.

VI. NATIVE LOW-BIT DEPLOYMENT

A. Latency beyond the float engine

Figure 4 reports observation-to-action latency with W4A4 projections in both backbone and expert. Desktop speedups are $1.66\text{--}2.22\times$ over eager PyTorch and $1.25\text{--}1.52\times$ over floating-point TensorRT. W4A4 further reduces latency by $10.3\text{--}15.3\%$ relative to FQ W8A8.

TABLE I

LIBERO SUCCESS RATE (%) AGAINST PRIOR LOW-BIT RECIPES. SPAT./OBJ.: SPATIAL/OBJECT SUITES; AVG.: MEAN OVER FOUR SUITES. Δ : CHANGE FROM BF16 IN PERCENTAGE POINTS, COMPUTED BEFORE ROUNDING. PROTOCOLS DIFFER BETWEEN CHECKPOINT BLOCKS.

Method	Spat.	Obj.	Goal	Long	Avg.	Δ
<i>GR00T N1.7</i> (NVIDIA per-suite checkpoints, 720-step cap)						
BF16 PyTorch (reference)	98.5	98.5	92.5	93.5	95.8	–
HoloQ-VLA [†]	94.0	98.5	89.5	90.0	93.0	–2.8
DuQuant [†]	97.0	97.5	95.0	83.5	93.3	–2.5
FQ W4A4 (ours)	98.5	97.0	95.5	88.5	94.9	–0.9
FQ W4A4 + o/d INT8 (ours)	96.5	98.5	94.0	93.5	95.6	–0.1
<i>$\pi_{0.5}$</i> (author-released checkpoint)						
BF16 PyTorch (reference)	98.5	99.0	97.5	93.5	97.1	–
HoloQ-VLA*	99.0	97.0	100.0	96.0 [§]	98.0	+0.9
DuQuant*	96.0	99.0	94.0	88.0	94.3	–2.9
QuantVLA variant*	94.0	98.0	80.0	56.0	82.0	–15.1
SmoothQuant*	83.0	88.0	40.0	26.0	59.3	–37.9
FQ W4A4 (ours)	98.0	99.5	96.0	95.5	97.3	+0.1
FQ W4A4 + o/d INT8 (ours)	99.5	99.0	98.5	94.0	97.8	+0.6

N1.7: 20 initial states per task, separate from P3; quantized rows share one harness on RTX 4070 Ti SUPER; BF16 uses the upstream harness on L4. [†]Our emulated implementation of the recipe, released with our code. Paired over 40 tasks, both FoldQuantVLA arms lead both recipes by 13–21 episodes ($p = 0.12\text{--}0.22$). $\pi_{0.5}$: BF16 and FoldQuantVLA rows use the released harness (20 trials per task, $K = 5$); BF16 (777/800) matches the rounded 97.1% float rate reported by HoloQ-VLA. *Reported by HoloQ-VLA [22]: full W4A4 including DiT attention, emulated, ten calibration trajectories, 10 trials per task (400 episodes). The QuantVLA row is a full-projection W4A4 variant, not its original selective W4A8 recipe. [§]Our re-run of their released pack in their harness: 94.0.

On Orin, speedups are $2.24\text{--}5.01\times$ over eager PyTorch and $1.20\text{--}1.33\times$ over the float engine. Compilation already accounts for 83–92% of the eager-to-W4A4 reduction; the remaining gain comes from the low-bit configuration. ModelOpt W4A16 AWQ is slower than the float engine on all four checkpoints: low-bit weight storage alone does not ensure batch-1 acceleration.

B. Cost of selective INT8

The o/d INT8 configuration adds 1.8, 1.3, and 0.4 ms to desktop W4A4 for the three GR00T checkpoints, and 1, 2, and 3 ms on Orin. Thus the N1.7 configuration used on the robot takes 120 ms rather than 119 ms, while remaining faster than W8A8 at 127 ms and float TensorRT at 146 ms. The cost is larger for $\pi_{0.5}$: 6.2 ms on desktop and 31 ms on Orin. Selective INT8 is therefore a checkpoint-dependent trade-off.

C. Storage and resident memory

For N1.6, weight storage falls from 3802 MB for floating-point TensorRT to 2104 MB for W8A8 and 1011 MB for W4A4; o/d INT8 uses 1189 MB. Complete serialized TensorRT engines occupy 5325 and 2525 MB for floating point and W4A4, respectively. Resident memory for float/W4A4 is 6349/4673 MiB when replaced PyTorch weights are never materialized, and 10609/8933 MiB on the shipped serving path.

VII. ACCURACY AND SELECTIVE PRECISION

A. Main closed-loop comparison

Table II reports closed-loop behavior for the five deployment configurations.

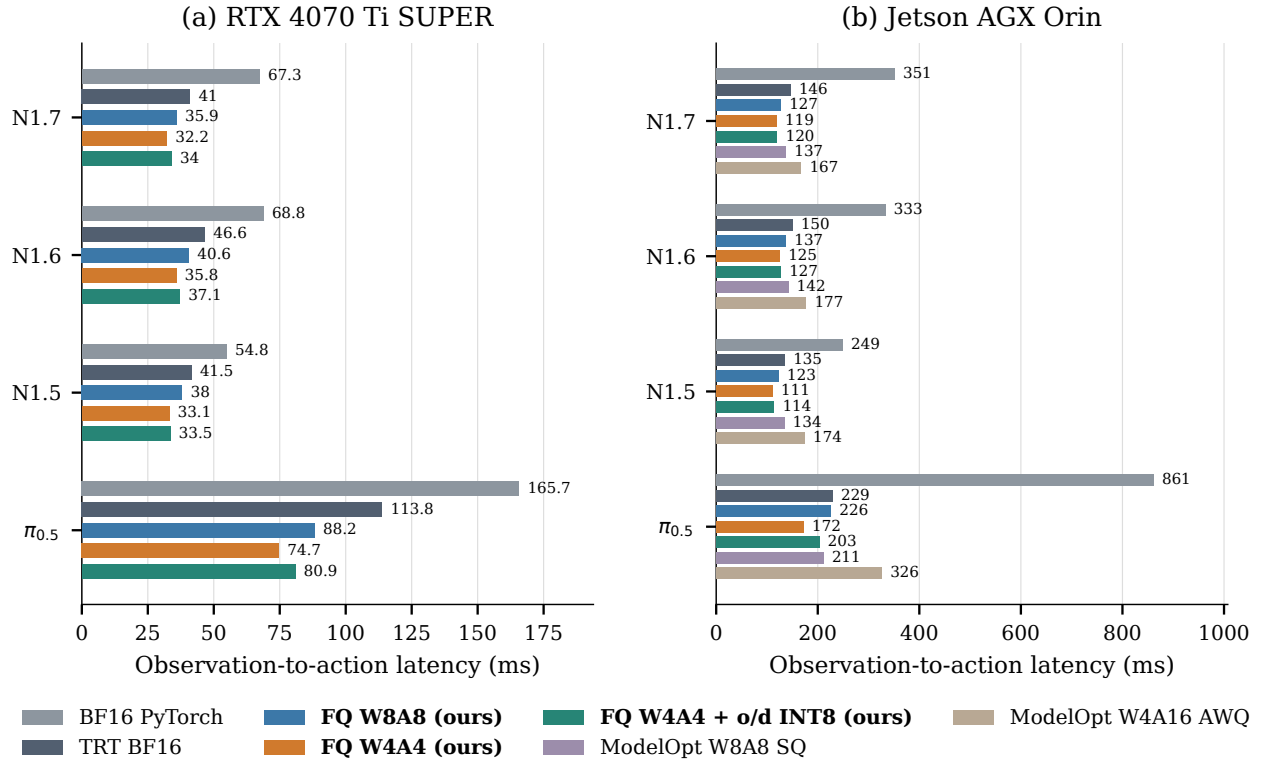


Fig. 4. Observation-to-action latency at batch 1 (lower is better). Panels share method colors; axes differ across platforms. Bar labels are milliseconds. Each checkpoint is compared with its own floating-point TensorRT engine. W4A4 provides an additional $1.25\text{--}1.52\times$ speedup on desktop and $1.20\text{--}1.33\times$ on Orin. Selective o/d INT8 trades some of this gain for action fidelity.

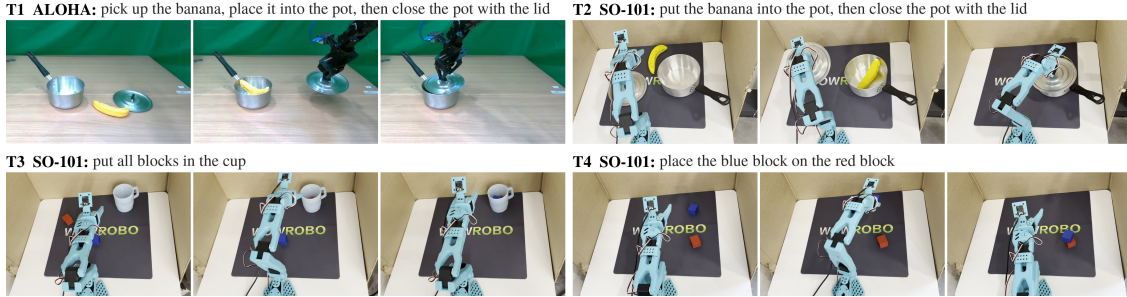


Fig. 5. Real-robot tasks, with successive frames from one successful episode per task. T1 uses single-arm ALOHA; T2–T4 use SO-101. T1/T2 require the banana in the pot and the lid placed on it; T2 does not require the lid to sit tightly. T3 requires every block inside the cup, and T4 requires the blue block resting on the red block. Frames illustrate tasks, not an arm-to-arm comparison.

TABLE II
LIBERO P3 SUCCESS (%); 800 EPISODES PER CELL, USING THE
DEPLOYMENT CONFIGURATIONS. PRESET-MATCHED SELECTIVE-PRECISION
COMPARISONS ARE REPORTED SEPARATELY IN THE TEXT.

Method	N1.7	N1.6	N1.5	$\pi_{0.5}$
BF16 PyTorch	96.25	96.38	86.38	96.50
TRT BF16	95.50	97.75	86.00	98.00
FQ W8A8 (ours)	95.38	95.75	87.12	97.38
FQ W4A4 (ours)	95.38	95.75	87.38	97.12
FQ W4A4 + o/d INT8 (ours)	95.00	96.62	87.00	97.62

To isolate the precision change in the o/d INT8 comparison,

we use controls with the same base preset and calibration rather than the main W4A4 row. N1.6 improves from 763 to 773/800 ($p = 0.17$); N1.7 from 757 to 760 ($p = 0.77$). N1.5 changes from 699 to 696 ($p = 0.78$), and $\pi_{0.5}$ from 777 to 781 ($p = 0.42$). These separate paired tests do not resolve a LIBERO gain from selective INT8. We therefore motivate it through fidelity and real-robot evidence, without claiming simulator superiority.

B. SimplerEnv Bridge

We also evaluate the released GR00T N1.6 Bridge checkpoint on seven WidowX tasks from the GR00T fork of SimplerEnv [35], [36] (Table III; 300-step cap, four executed

TABLE III

SIMPLERENV BRIDGE (WIDOWX) WITH GR00T N1.6: SR (%) OVER 200 EPISODES PER TASK ON RTX 4070 T1 SUPER. FQ DENOTES OUR METHOD.

Task	BF16	FQ W8A8 (ours)	FQ W4A4 (ours)	FQ W4A4 + o/d INT8 (ours)
Spoon on towel	63.5	63.5	66.0	75.5
Carrot on plate	60.5	59.5	70.5	66.5
Eggplant in basket	92.5	89.5	58.0	70.5
Stack cube	6.0	2.5	7.5	5.5
Eggplant in sink	43.0	40.0	63.0	50.0
Close drawer	72.0	64.5	82.0	89.5
Open drawer	98.0	96.5	81.5	92.0
Mean	62.2	59.4	61.2	64.2

actions). Mean success is similar in these runs: 61.2% for W4A4 and 64.2% for o/d INT8, against 62.2% for BF16 (paired over tasks, $p = 0.89$ and 0.70). These tests do not establish equivalence. The aggregate masks larger per-task differences: the mean absolute per-task difference between W4A4 and BF16 is 13.6 percentage points, with its largest loss on the basket task (-34.5). Selective INT8 reduces it to 10.1 points and improves open drawer ($81.5 \rightarrow 92.0\%$) and basket ($58.0 \rightarrow 70.5\%$).

C. Held-out action fidelity

On P2, retaining o/d at INT8 raises the minimum decoded-action cosine on all four checkpoints: from 0.461 to 0.850 for N1.6, 0.802 to 0.912 for N1.7, 0.970 to 0.973 for N1.5, and 0.847 to 0.9985 for $\pi_{0.5}$. Their median maximum absolute coordinate errors fall from 0.074 to 0.039 for N1.6 and from 0.099 to 0.058 for N1.7. W8A8 remains closer to the float reference on these metrics.

A few observations dominate the mean cosine deficit ($1 - \cos$): the worst W4A4 observation contributes 65% for N1.6 and 43% for N1.7. Across 19 arms with P1/P3 measurements (FQ W8A8/W4A4 and ModelOpt SQ/AWQ on four checkpoints, plus three o/d INT8 arms), cosine is at least 0.998 and its rank correlation with success differences is not significant (Spearman $\rho = 0.20$, $p = 0.42$). Shared checkpoints and references make this pooled correlation descriptive.

D. Design ablations

In Table IV, the external per-tensor control also changes calibration and execution, so it does not isolate scale granularity. The mismatched fold was tested offline only. FWHT and fold-before are kernel-speed choices with offline cosine changes of at most 2×10^{-4} . GPTQ and o/d INT8 improve fidelity; none of these paired design comparisons resolves a closed-loop difference over 800 episodes.

VIII. REAL-ROBOT EVALUATION

We deploy the Orin engines on single-arm ALOHA and SO-101, using the tasks in Fig. 5. GR00T N1.7 uses one fine-tune per embodiment. Each arm is tested for 20 episodes per task, interleaved within a session with matched object placements.

Selective INT8 improves observed robot outcomes. Tables V and VI show an improvement over uniform W4A4 on

TABLE IV

OFFLINE FIDELITY AND LIBERO ABLATIONS. DESIGN COMPARISONS REPORT CHANGES IN SUCCESSES OUT OF 800 (FIRST MINUS SECOND), WITH PAIRED TASK-LEVEL p VALUES IN PARENTHESES. CALIBRATION CONTROLS ARE SEPARATED BELOW; -: NOT EVALUATED.

First vs. second	Offline	N1.6	N1.7
Consistent vs. mismatched fold	cos .9997 vs. .9923	–	–
GPTQ vs. RTN	deficit $-25\%/ -37\%$ ^a	-6 (.45)	$+7$ (.23)
Butterfly vs. composite rot.	$ \Delta \cos \leq 2 \times 10^{-4}$	$+10$ (.18)	-1 (.87)
Fold-before vs. after	$ \Delta \cos \leq 2 \times 10^{-4}$	$+4$ (.66)	-11 (.19)
o/d INT8 vs. W4A4	deficit -65%	$+10$ (.17)	$+3$ (.77)
<i>Calibration controls (W8A8)</i>			
Per-token vs. per-tensor	LLM cos .998 vs. .991	–	–
ModelOpt per-tensor vs. BF16	–	-41.0 pp ^b	-5.5 pp ^b

Offline: action-module cosine for the fold comparison, decoded-action fidelity for the remaining design comparisons. Deficit is $1 - \cos$; its changes are relative percentages. ^aN1.6/N1.7; other offline entries are N1.6. ^bPercentage-point SR changes: external ModelOpt per-tensor max calibration with no smoothing versus BF16 (800 episodes; McNemar $p < 0.001$). This control also changes the calibration and execution pipeline.

TABLE V

N1.7 ON SO-101: SR (%) OVER 20 TRIALS PER TASK AND ARM ON ORIN. T2–T4 ARE DEFINED IN FIG. 5; AVG. IS THEIR MEAN. [‡]STOPPED AFTER T4 BECAUSE JERKY MOTION RISKED THE HARDWARE; N/A: NOT EVALUATED.

Method	T2	T3	T4	Avg.
BF16 PyTorch	90.0	100.0	70.0	86.7
TRT BF16	90.0	100.0	75.0	88.3
ModelOpt W8A8 SQ [‡]	n/a	n/a	30.0	n/a
ModelOpt W4A16 AWQ	n/a	n/a	40.0	n/a
FQ W8A8 (ours)	95.0	100.0	70.0	88.3
FQ W4A4 (ours)	85.0	95.0	65.0	81.7
FQ W4A4 + o/d INT8 (ours)	95.0	100.0	80.0	91.7

TABLE VI
N1.7 ON ALOHA: SR (%), 20 TRIALS PER ARM. T1 AS IN FIG. 5.

Method	T1: banana in pot, lid
BF16 PyTorch	90.0
TRT BF16	95.0
FQ W8A8 (ours)	85.0
FQ W4A4 (ours)	75.0
FQ W4A4 + o/d INT8 (ours)	95.0

TABLE VII
 $\pi_{0.5}$ ON SO-101 (ITS OWN FINE-TUNE): SR (%), 20 TRIALS PER ARM. T4 AS IN FIG. 5.

Method	T4: blue block on red block
BF16 PyTorch	80.0
TRT BF16	100.0
FQ W4A4 (ours)	85.0

every N1.7 task. Pooled success is 92.5% for o/d INT8, 80.0% for W4A4, 87.5% for W8A8 and BF16 PyTorch, and 90.0% for float TensorRT. The o/d INT8 gain costs 1 ms on Orin; these observed rates do not establish equivalence to W8A8 or BF16.

On logged ALOHA and SO-101 observations, the median action cosine against BF16 is 0.99962 and 0.99961 for N1.7 W4A4, respectively; selective INT8 raises these medians to 0.99975 and 0.99988.

ModelOpt W8A8 SmoothQuant uses static per-tensor scales and reached 30.0% on T4 versus 70.0% for BF16 and FQ W8A8. Jerky motion risked the hardware, so testing stopped at T4 and this arm is excluded from the aggregate. Its median SO-101 action cosine is 0.99892 despite poor task performance. FQ W8A8 also uses rotations, precluding attribution solely to

per-token scaling. ModelOpt W4A16 AWQ reaches 40.0% on T4, with mean action cosine 0.99980 and minimum 0.99882; it is also excluded from the four-task aggregate.

For $\pi_{0.5}$ on SO-101 T4 (Table VII), W4A4 success is close to its own BF16 reference despite a median action cosine of 0.98994, substantially below N1.7's. A fixed cosine threshold therefore does not transfer automatically between checkpoints.

IX. LIMITATIONS AND REPRODUCIBILITY

Architecture dependence. Consistent folding does not guarantee useful W4A4 accuracy for every VLA. Archived exploratory LIBERO campaigns (800 episodes per arm) give SmolVLA [37] 27.1% versus BF16's 71.1% (217 versus 569 successes), and Evo-1 [38] 81.0% versus 91.6% (648 versus 733). These are configuration-specific failures; calibration differences limit attribution of Evo-1's entire deficit to bit width. We have not established that o/d INT8 restores either architecture.

Runtime and hardware. Projection coverage does not guarantee a speedup. Native INT4 is measured on Ada and Orin; the inspected H100 backend lowers four-bit operands to INT8 arithmetic. H100 success therefore establishes neither native-INT4 latency nor bitwise cross-device equivalence.

Statistical and behavioral scope. Most simulator arms have one campaign over 40 tasks, and matched initial states do not eliminate serving noise. Physical trials number only 20 per task; $\pi_{0.5}$ is tested on one task. An exploratory two-sided Fisher exact test on the pooled N1.7 robot counts (74/80 versus 64/80) gives $p = 0.037$, but ignores matched placements and task/session dependence. We therefore report the robot gain as an observed difference. P1/P2 each use 32 observations, cosine ignores magnitude, and binary success omits smoothness and contact forces.

Attribution and reproducibility. N1.6/N1.7 deployment comparisons include calibration changes; only preset-matched controls isolate precision. The two o/d types are evaluated together, leaving their individual effects unresolved. One desktop timing run does not quantify run-to-run variability. Reproduction requires each result's checkpoint, calibration samples, transform preset, and evaluation protocol.

X. CONCLUSION

FoldQuantVLA carries consistent activation coordinates from calibration to native low-bit execution in both language backbone and action expert. W4A4 reduces latency beyond compiled float engines on desktop and Orin. Selective o/d INT8 improves held-out fidelity and raises observed N1.7 robot success from 80.0% to 92.5% for 1 ms on Orin, though its LIBERO gains remain unresolved and its cost is larger for $\pi_{0.5}$. Deployment requires checkpoint-specific latency and closed-loop evaluation.

REFERENCES

- [1] NVIDIA *et al.*, "GR00T N1: An Open Foundation Model for Generalist Humanoid Robots," 2025, arXiv:2503.14734.
- [2] Physical Intelligence *et al.*, " $\pi_{0.5}$: a Vision-Language-Action Model with Open-World Generalization," 2025, arXiv:2504.16054.
- [3] G. Xiao *et al.*, "SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models," in *ICML*, 2023.
- [4] S. Ashkboos *et al.*, "QuaRot: Outlier-Free 4-Bit Inference in Rotated LLMs," in *NeurIPS*, 2024.
- [5] Z. Liu *et al.*, "SpinQuant: LLM quantization with learned rotations," in *ICLR*, 2025.
- [6] H. Lin *et al.*, "DuQuant: Distributing Outliers via Dual Transformation Makes Stronger Quantized LLMs," in *NeurIPS*, 2024.
- [7] P. Czako, G. Kertész, and S. Szénási, "SmoothRot: Combining Channel-Wise Scaling and Rotation for Quantization-Friendly LLMs," 2025, arXiv:2506.05413.
- [8] E. Frantar *et al.*, "GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers," in *ICLR*, 2023.
- [9] S. Ashkboos *et al.*, "SliceGPT: Compress Large Language Models by Deleting Rows and Columns," in *ICLR*, 2024.
- [10] Y. Sun *et al.*, "FlatQuant: Flatness Matters for LLM Quantization," in *ICML*, 2025.
- [11] X. Hu *et al.*, "OstQuant: Refining Large Language Model Quantization with Orthogonal and Scaling Transformations for Better Distribution Fitting," in *ICLR*, 2025.
- [12] J. Lin *et al.*, "AWQ: Activation-aware Weight Quantization for On-Device LLM Compression and Acceleration," in *MLSys*, 2024.
- [13] Y. Zhao *et al.*, "Atom: Low-bit Quantization for Efficient and Accurate LLM Serving," in *MLSys*, 2024.
- [14] Y. Lin *et al.*, "QServe: W4A8KV4 Quantization and System Co-design for Efficient LLM Serving," in *MLSys*, 2025.
- [15] M. Li *et al.*, "SVDQuant: Absorbing Outliers by Low-Rank Components for 4-Bit Diffusion Models," in *ICLR*, 2025.
- [16] Z. Zheng, X. Song, and C. Liu, "MixLLM: LLM Quantization with Global Mixed-precision between Output-features and Highly-efficient System Design," in *MLSys*, 2026.
- [17] Z. Yao *et al.*, "HAWQ-V3: Dyadic Neural Network Quantization," in *ICML*, 2021.
- [18] J. Zhang *et al.*, "QuantVLA: Scale-Calibrated Post-Training Quantization for Vision-Language-Action Models," in *CVPR*, 2026.
- [19] Y. Xu *et al.*, "QVLA: Not All Channels Are Equal in Vision-Language-Action Model's Quantization," in *ICLR*, 2026.
- [20] A. Akbari *et al.*, "ActQuant: Sub-4-bit Action-Guided Quantization for Vision-Language-Action Models," 2026, arXiv:2605.24011.
- [21] Z. Zheng *et al.*, "DyQ-VLA: Temporal-Dynamic-Aware Quantization for Embodied Vision-Language-Action Models," 2026, arXiv:2603.07904.
- [22] X. Wang *et al.*, "HoloQ-VLA: Uniform W4A4 Quantization of Vision-Language-Action Models," 2026, arXiv:2605.28803v3.
- [23] S. Lee *et al.*, "AccuQuant: Simulating Multiple Denoising Steps for Quantizing Diffusion Models," 2025, arXiv:2510.20348.
- [24] S. Ryu, M. Salzmann, and S. Javed, "Q-Drift: Quantization-Aware Drift Correction for Diffusion Model Sampling," 2026, arXiv:2603.18095.
- [25] Y. Li *et al.*, "GPTAQ: Efficient Finetuning-Free Quantization for Asymmetric Calibration," in *ICML*, 2025.
- [26] S. Zhang *et al.*, "Qronos: Correcting the Past by Shaping the Future... in Post-Training Quantization," in *ICLR*, 2026.
- [27] NVIDIA, "TensorRT Developer Guide (v10.3)," 2024, weight-only quantization section.
- [28] K. Agarwal *et al.*, "HadaCore: Tensor Core Accelerated Hadamard Transform Kernel," 2024, arXiv:2412.08832.
- [29] H. Wang, S. Ma, and F. Wei, "BitNet a4.8: 4-bit Activations for 1-bit LLMs," arXiv:2411.04965, 2024.
- [30] NVIDIA, "NVIDIA Isaac GR00T N1.7," 2026, model card, accessed Sep. 15, 2026.
- [31] NVIDIA, "NVIDIA Isaac GR00T N1.6," 2025, model card, accessed Sep. 15, 2026.
- [32] NVIDIA, "NVIDIA Isaac GR00T N1.5: an open foundation model for humanoid robots," 2025, model card, accessed Sep. 15, 2026.
- [33] B. Liu *et al.*, "LIBERO: Benchmarking Knowledge Transfer for Lifelong Robot Learning," in *NeurIPS*, 2023.
- [34] NVIDIA, "NVIDIA Model Optimizer," accessed Sep. 15, 2026.
- [35] X. Li *et al.*, "Evaluating Real-World Robot Manipulation Policies in Simulation," arXiv:2405.05941, 2024.
- [36] NVIDIA, "Isaac GR00T: SimplerEnv Evaluation," accessed Sep. 21, 2026.
- [37] M. Shukor *et al.*, "SmolVLA: A Vision-Language-Action Model for Affordable and Efficient Robotics," 2025, arXiv:2506.01844.
- [38] T. Lin *et al.*, "Evo-1: Lightweight Vision-Language-Action Model with Preserved Semantic Alignment," 2025, arXiv:2511.04555.